

Report Progetto CSI

Giacomo Galli, Emilio Gigante, Gabriele Scarpelli

AA - 2023/2024

Indice

1	Introduzione	3
2	Sistema Fisico	3
2.1	Modello nominale	4
2.2	Linearizzazione	4
2.3	Modello attuatori	5
2.4	Schema a blocchi	6
3	Modello delle incertezze	7
3.1	Incertezze sul sistema	7
3.2	Incertezze sugli attuatori	8
4	Forma di Doyle	11
4.1	Ingressi esogeni e uscite di prestazione	11
4.1.1	Generazione peso di prestazione sull'errore	11
4.1.2	Generazione peso di prestazione sull'ingresso	12
4.1.3	Generazione peso di prestazione sull'uscita	12
4.2	Ottenimento della matrice P	13
5	Controllore H-infinito	15
5.1	Sintesi	15
5.2	Mu - Analisi	15
5.2.1	Stabilità nominale	16
5.2.2	Performance nominali	16
5.2.3	Robusta stabilità	17
5.2.4	Robusta performance	18
5.3	Risultati simulazione	20
5.3.1	Sistema linearizzato, senza disturbi	20
5.3.2	Sistema linearizzato, con disturbi	21
5.3.3	Sistema "reale", senza disturbi	22
5.3.4	Sistema "reale", con disturbi	23

6	Controllore D-K	24
6.1	Sintesi	24
6.2	Mu - Analisi	26
6.2.1	Stabilità nominale	26
6.2.2	Performance nominali	27
6.2.3	Robusta stabilità	27
6.2.4	Robusta performance	28
6.3	Risultati simulazione	30
6.3.1	Sistema linearizzato, senza disturbi	30
6.3.2	Sistema linearizzato, con disturbi	31
6.3.3	Sistema "reale", senza disturbi	32
6.3.4	Sistema "reale", con disturbi	33
7	Controllore LQG	34
7.1	Sintesi	36
7.2	LQG con integratore	38
7.3	Risultati simulazione	39
7.3.1	Sistema linearizzato, senza disturbi	39
7.3.2	Sistema linearizzato, con disturbi	40
7.3.3	Sistema linearizzato, con disturbo a gradino su ingresso	41
7.3.4	Sistema linearizzato, con disturbo a gradino su ingresso, con azione integrale di controllo	42
7.3.5	Sistema "reale", senza disturbi	43
8	Extra	44
8.1	Calcolo manuale della matrice N_G	44
8.2	Incertezze parametriche sugli attuatori	46
8.3	Sintesi controllore D-K con implementazione manuale dell'algoritmo	47
8.4	Riduzione dell'ordine del controllore DK	50

1 Introduzione

L'obiettivo del progetto presentato è la sintesi di controllori da applicare al sistema fisico assegnato, tenendo conto dell'incertezza sui parametri del sistema e dell'azione di disturbi.

A tal fine, sono stati realizzati differenti controllori seguendo alcune delle metodologie viste durante il corso. In particolare, sono stati sintetizzati controllori utilizzando la sintesi H-infinito, la D-K iteration e il controllo di tipo LQG (con e senza azione integrale).

Nella seguente relazione sono quindi presentate in dettaglio le tecniche usate per modellare il sistema, rappresentare le incertezze e le performance richieste e realizzare i suddetti controllori, e vengono esposti i risultati ottenuti applicando i regolatori al sistema.

2 Sistema Fisico

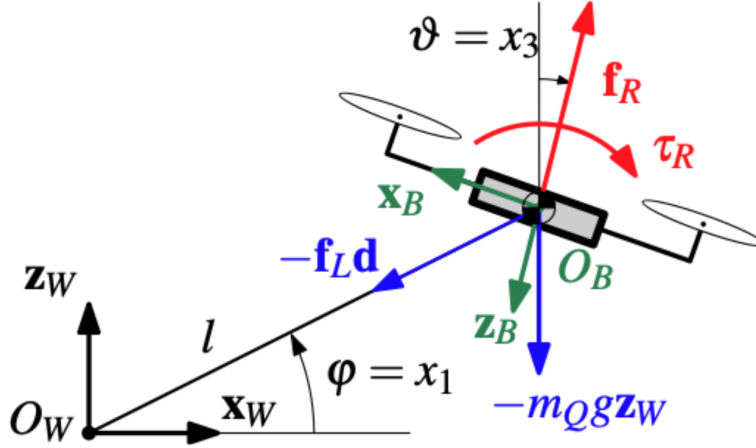


Figura 1: Sistema fisico assegnato

Il sistema fisico in esame¹ è costituito da un drone incernierato tramite una sbarra, e soggetto alle equazioni dinamiche:

$$M\ddot{q} + G(q) = Q(q)u$$

con la configurazione del sistema data da

$$q = \begin{bmatrix} \phi \\ \theta \end{bmatrix}$$

dove ϕ è l'angolo della sbarra rispetto all'asse X_W e θ è l'angolo tra Z_W e $-Z_B$, che rappresenta l'orientazione del drone. Si ha inoltre:

$$M = \begin{bmatrix} J_\phi & 0 \\ 0 & J_\theta \end{bmatrix} \quad G = \begin{bmatrix} m g l \cos(\phi) \\ 0 \end{bmatrix} \quad Q = \begin{bmatrix} l \cos(\phi + \theta) & 0 \\ 0 & 1 \end{bmatrix} \quad u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} f_R \\ \tau_R \end{bmatrix}$$

¹Tratto dal seguente articolo: <https://homepages.laas.fr/~afranchi/robotics/sites/default/files/1603.07567-v2.pdf>

dove m rappresenta la massa del drone, J_θ e $J_\phi = ml^2$ i momenti d'inerzia relativi a ϕ e θ , e l la lunghezza della sbarra. Gli ingressi del sistema sono f_R e τ_R , ossia la forza (in direzione $-Z_B$) e la coppia applicate al drone.

Come obiettivo di controllo, è stato considerato di voler mantenere il drone in equilibrio nella posizione

$$\begin{bmatrix} \phi \\ \theta \end{bmatrix} = \begin{bmatrix} \frac{\pi}{4} \\ 0 \end{bmatrix}$$

con velocità angolari nulle.

2.1 Modello nominale

Per prima cosa, prendendo come stato

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} \phi \\ \theta \\ \omega_\phi \\ \omega_\theta \end{bmatrix}$$

e come ingresso

$$u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} f_R \\ \tau_R \end{bmatrix}$$

è stata scritta la dinamica non lineare in forma di stato:

$$\begin{cases} \dot{x}_1 = x_3 \\ \dot{x}_2 = x_4 \\ \dot{x}_3 = -\frac{m l g}{J_\phi} \cos(x_1) + \frac{l}{J_\phi} \cos x_1 + x_2 u_1 \\ \dot{x}_4 = \frac{1}{J_\theta} u_2 \end{cases}$$

Come uscita, si è ipotizzato di avere a disposizione ϕ e θ , avendo verificato che il sistema (linearizzato intorno all'equilibrio desiderato) risulta essere completamente osservabile con tale output. Le equazioni relative all'uscita sono:

$$\begin{cases} y_1 = x_1 \\ y_2 = x_2 \end{cases}$$

2.2 Linearizzazione

La dinamica del sistema è stata quindi linearizzata intorno allo stato di equilibrio

$$\bar{x} = \begin{bmatrix} \pi/4 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

con relativo ingresso di equilibrio

$$\bar{u} = \begin{bmatrix} mg \\ 0 \end{bmatrix}$$

e riscrivendo di conseguenza il sistema nella forma:

$$\begin{cases} \dot{\tilde{x}} = A\tilde{x} + B\tilde{u} \\ \tilde{y} = C\tilde{x} + D\tilde{u} \end{cases}$$

con $\tilde{x} = x - \bar{x}$, $\tilde{u} = u - \bar{u}$ e $\tilde{y} = y - C\bar{x}$, laddove:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{m l g}{J_\phi \sqrt{2}} & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ \frac{l}{J_\phi \sqrt{2}} & 0 \\ 0 & \frac{1}{J_\theta} \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Si è pertanto potuta ricavare la matrice di trasferimento relativa al sistema

$$G(s) = C(s \cdot I - A)^{-1}B + D = ss(A, B, C, D)$$

ottenendo come risultato:

$$G(s) = \begin{bmatrix} \frac{l}{\sqrt{2}J_\phi s^2} & -\frac{m l g}{\sqrt{2}J_\phi J_\theta s^4} \\ 0 & \frac{1}{J_\theta s^2} \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}m l s^2} & -\frac{g}{\sqrt{2}l J_\theta s^4} \\ 0 & \frac{1}{J_\theta s^2} \end{bmatrix}$$

2.3 Modello attuatori

Gli ingressi al sistema vengono dati mediante l'uso di attuatori, la cui funzione di trasferimento è considerata della forma:

$$G_a = \frac{k_a}{T_a s + 1}$$

La matrice di trasferimento relativa agli attuatori è stata quindi considerata come una matrice 2x2 diagonale, con i due elementi sulla diagonale uguali tra loro e della forma della funzione di trasferimento vista sopra.

2.4 Schema a blocchi

Complessivamente, lo schema a blocchi rappresentativo del sistema linearizzato risulta essere:

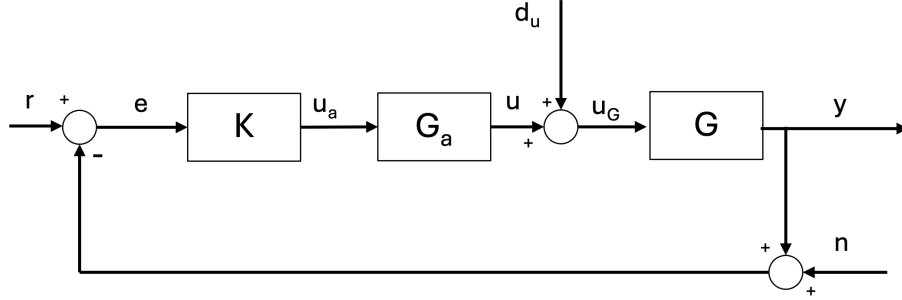


Figura 2: Schema a blocchi del sistema linearizzato nominale

Possiamo perciò ricavare alcune delle matrici caratteristiche che legano tra loro le varie grandezze che appaiono nello schema:

$$\begin{aligned} L_i &= KG_aG & S_i &= (I + L_i)^{-1} & T_i &= S_i L_i \\ L_o &= GG_aK & S_o &= (I + L_o)^{-1} & T_o &= S_o L_o \\ L_m &= G_aKG & S_m &= (I + L_m)^{-1} & T_m &= S_m L_m \end{aligned}$$

ed esplicitare tali relazioni:

$$\begin{cases} y = T_o(r - n) + S_o G d_u \\ e = S_o(r - n) - S_o G d_u \\ u = S_m G_a K(r - n) - T_m d_u \end{cases}$$

Si osservi che, nel sistema linearizzato, la variabile di stato è in realtà $\tilde{x}$, ossia la differenza tra la posizione effettiva e quella di equilibrio. Il problema è quindi di regolazione, con riferimento $r = 0$, in quanto si vorrebbe idealmente $x = \bar{x}$.

3 Modello delle incertezze

Sono stati utilizzati due modelli diversi per rappresentare l'incertezza relativa al sistema e agli attuatori: per il sistema fisico è stato usato un modello di incertezza parametrica (reale) mentre per gli attuatori è stata presa un'incertezza concentrata (complessa), modellata come moltiplicativa in ingresso.

3.1 Incertezze sul sistema

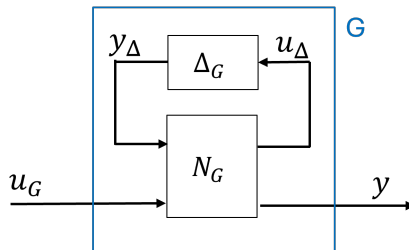


Figura 3: Schema dell'incertezza parametrica del sistema

I parametri incerti che intervengono nella dinamica del sistema sono la massa del drone m , la lunghezza della sbarra l e il momento d'inerzia baricentrico del drone J_θ . I valori nominali relativi a tali parametri sono stati presi pari a

$$\bar{m} = 1 \text{ kg} \quad \bar{l} = 2 \text{ m} \quad \bar{J}_\theta = 0.25 \text{ kg m}^2$$

come riportato nella traccia del progetto assegnata.

I valori effettivi di questi parametri sono stati invece presi come

$$m = \bar{m} \cdot (1 + \alpha_m \delta_m)$$

$$l = \bar{l} \cdot (1 + \alpha_l \delta_l)$$

$$J_\theta = \bar{J}_\theta \cdot (1 + \alpha_{J_\theta} \delta_{J_\theta})$$

con $|\delta_m|$, $|\delta_l|$ e $|\delta_{J_\theta}| < 1$. Per quantificare l'incertezza sui vari parametri abbiamo utilizzato valori coerenti con quelli usati durante il progetto di Identificazione dei Sistemi Incerti, assumendo pertanto:

$$\alpha_m = 0.05 \quad \alpha_l = 0.036 \quad \alpha_{J_\theta} = 0.015$$

Questi parametri incerti sono stati generati tramite il comando *ureal* e utilizzati per scrivere le matrici della forma di stato (A, B, C, D). Fatto ciò, è stato generato tramite il comando *ss* il sistema incerto G e successivamente, utilizzando il comando *lftdata*, è stata estratta da G la matrice Δ_G che descrive l'incertezza sul sistema e la matrice N_G tale per cui $G = \text{lft}(\Delta_G, N_G)$.

```
[N_G, Delta_G, blkstruct_G] = lftdata(G);
```

```
>> Delta_G
```

```
Uncertain matrix with 3 rows and 3 columns.
```

The uncertainty consists of the following blocks:

J_theta: Uncertain real, nominal = 0.25, variability = [-0.009,0.009], ...
1 occurrences

l: Uncertain real, nominal = 2, variability = [-0.03,0.03], 1 occurrences

m: Uncertain real, nominal = 1, variability = [-0.05,0.05], 1 occurrences

3.2 Incertezze sugli attuatori

Dato che la matrice di trasferimento G_a relativa agli attuatori viene assunta diagonale, con elementi diagonali identici, è stato preso un unico modello per entrambi i canali. Il peso w_{ai} relativo all'incertezza è stato perciò generato lavorando nel caso SISO, e poi "sdoppiato" in modo da realizzare una matrice di peso W_a 2x2.

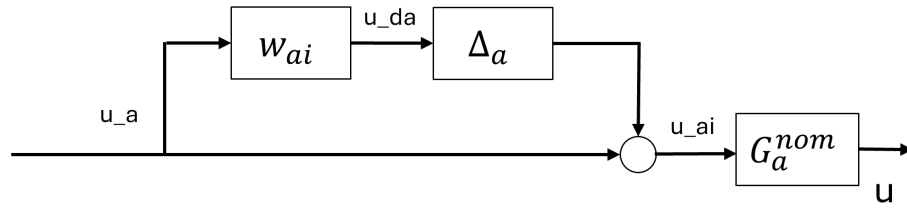


Figura 4: Modello di incertezza su G_a

I valori nominali dei parametri degli attuatori sono stati presi arbitrariamente come

$$\bar{k}_a = 1 \quad \bar{T}_a = 0.1 \text{ s}$$

mentre i parametri effettivi sono stati considerati come

$$k_a = \bar{k}_a \cdot (1 + \alpha_{k_a} \delta_{k_a})$$

$$T_a = \bar{T}_a \cdot (1 + \alpha_{T_a} \delta_{T_a})$$

con $|\delta_{k_a}|$ e $|\delta_{T_a}| < 1$, prendendo

$$\alpha_{k_a} = 0.05 \quad \alpha_{T_a} = 0.05$$

Per ottenere il peso w_{ai} , si generano un certo numero di realizzazioni della funzione di trasferimento G_a , e si calcola la differenza relativa tra esse e la funzione di trasferimento nominale. Dopodichè, si fa un grafico di tutte queste differenze, e si sceglie un peso che sia maggiore di ciascuna di esse. Si noti che in questo caso, a differenza dell'incertezza relativa al sistema fisico, il blocco Δ_a risulta complesso.

```
% Risposta in frequenza del sistema nominale
```

```
Ga_nom_frd = frd(Ga_nom_tf, omega);
```

```
% Genero N_realizzazioni del sistema incerto e trovo la differenza relativa
```

```
% con quello nominale
```

```

for i = 1:N_realizzazioni

    % Realizzazione del sistema
    Ga_p = tf(usample(Ga, 1));
    Ga_p_frd = frd(Ga_p, omega);

    % Differenza relativa
    Ga_rel_diff = (Ga_p_frd - Ga_nom_frd) / Ga_nom_frd;
    bodemag(Ga_rel_diff, 'c--', omega)
    hold on
end
grid on

% Genero un peso che "stia sopra" tutte le differenze relative trovate

num_punti = 5;          % Numero punti da prendere sul grafico
ord = 1;                % Ordine del peso generato

[freq,resp_dB] = ginput(num_punti);

% Riconverto i valori espressi in dB
resp = zeros(1,num_punti);
for i = 1:num_punti
    resp(i) = 10^(resp_dB(i) / 20);
end

wai_frd = frd(resp,freq);
wai_ss = fitmagfrd(wai_frd,ord);

bodemag(tf(wai_ss), 'r-')

% Peso sull'incertezza degli attuatori
wai = blkdiag(tf(wai_ss),tf(wai_ss));

```

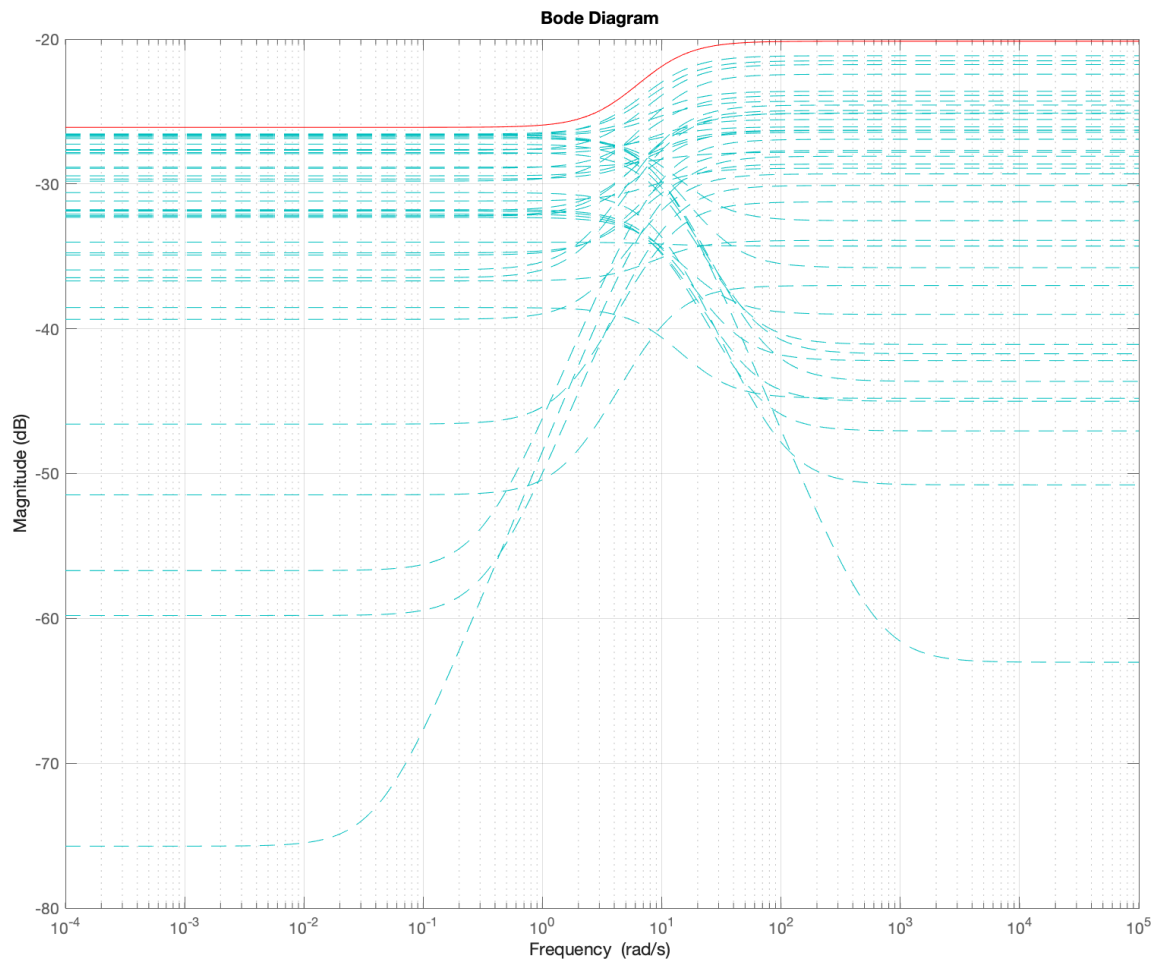


Figura 5: Generazione del peso relativo all'incertezza sugli attuatori (in rosso)

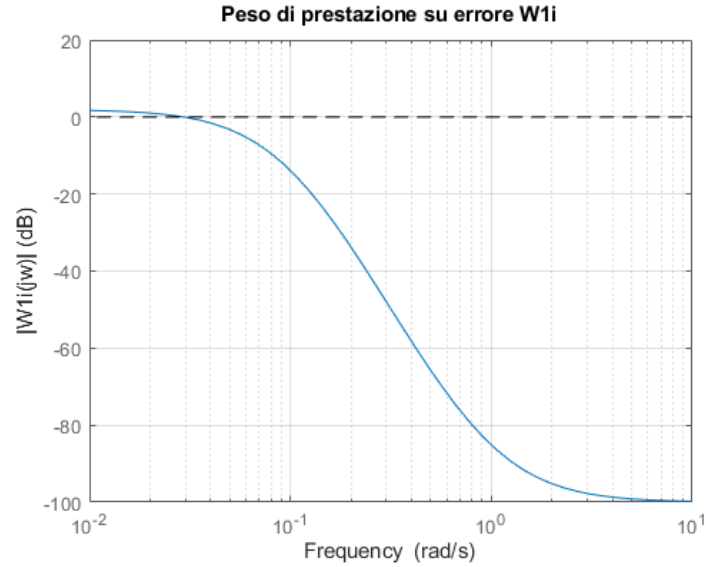


Figura 7: Diagramma del modulo di W_{1i} (peso di prestazione per l'errore)

- $\frac{1}{M} = \frac{1}{10^5}$: è il valore in alta frequenza del peso.
- $\frac{1}{a} = \frac{1}{0.8}$: è il valore in bassa frequenza del peso. Più alto è questo valore e più "peso" diamo al fatto di tenere S bassa in bassa frequenza.
- $N = 5$: è l'ordine del peso.

Con i valori elencati qua sopra otteniamo il peso di figura 7.

La matrice W_1 è stata ottenuta semplicemente come:

```
W1 = blkdiag(W1i, W1i);
```

4.1.2 Generazione peso di prestazione sull'ingresso

Per questo peso abbiamo selezionato un filtro passa alto del primo ordine:

$$W_{2i} = \frac{s}{s + \omega_C}$$

dove $\omega_C = 1000 \text{ rad/s}$ è la frequenza dalla quale cominciamo a pesare l'energia di controllo.

```
W2 = blkdiag(W2i, W2i);
```

4.1.3 Generazione peso di prestazione sull'uscita

Per il nostro progetto non è mai risultato necessario pesare l'uscita y oltre al peso sull'errore e e il controllo u . Pertanto, abbiamo scelto un peso nullo:

$$W_{3i} = 0$$

```
W3 = blkdiag(W3i, W3i);
```

4.2 Ottenimento della matrice P

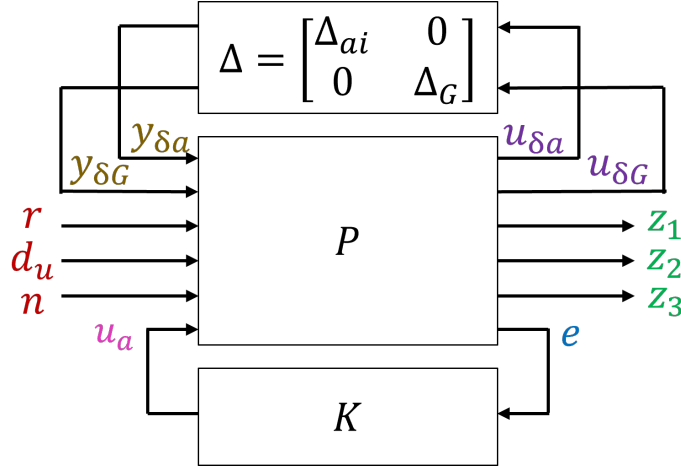


Figura 8: Schema a blocchi della forma DPK

Per portarsi nella forma ΔPK basta individuare quelli che saranno gli ingressi e le uscite della matrice P , come è stato fatto in modo piuttosto esaustivo in figura 8, dove i segnali in questione sono osservabili nello schema a blocchi all'inizio di questa sezione in figura 6.

A livello di codice MatLabTM, si sono modellati i vari componenti dello schema a blocchi, e poi, tramite il comando *connect*, abbiamo calcolato la matrice P :

```
% Ingressi e uscite di N_G
N_G.u(1) = {'y_D_G(1)'};
N_G.u(2) = {'y_D_G(2)'};
N_G.u(3) = {'y_D_G(3)'};
N_G.u(4) = {'u_G(1)'};
N_G.u(5) = {'u_G(2)'};

N_G.y(1) = {'u_D_G(1)'};
N_G.y(2) = {'u_D_G(2)'};
N_G.y(3) = {'u_D_G(3)'};
N_G.y(4) = {'y(1)'};
N_G.y(5) = {'y(2)'};

% Ingressi e uscite di Ga
Ga_nom.u = 'u_ai';
Ga_nom.y = 'u';

% Ingressi e uscite di wai
wai.u = 'u_a';
```

```

wai.y = 'u_D_a';

% Ingressi e uscite di W1
W1.u = 'e';
W1.y = 'z1';

% Ingressi e uscite di W2
W2.u = 'u_a';
W2.y = 'z2';

% Ingressi e uscite di W3
W3.u = 'y';
W3.y = 'z3';

% Nodi sommatori
Sum1 = sumblk('y_n = y + n',2);
Sum2 = sumblk('e = r - y_n',2);
Sum3 = sumblk('u_G = u + d_u',2);
Sum4 = sumblk('u_ai = u_a + y_D_a', 2);

% Uso comando connect per calcolare P
P = connect(N_G, Ga_nom, wai, W1, W2, W3, Sum1, Sum2, Sum3, Sum4,...
    {'y_D_a', 'y_D_G', 'r', 'd_u', 'n', 'u_a'},...
    {'u_D_a', 'u_D_G', 'z1', 'z2', 'z3', 'e'});

```

5 Controllore H-infinito

Il primo controllore realizzato è stato un controllore di tipo $\mathcal{H}_\infty$ sintetizzato tramite il comando *hinfsyn*.

5.1 Sintesi

Se consideriamo il sistema rappresentato nella forma $\Delta - P - K$ in figura 8, questo controllore cerca di minimizzare la norma ∞ della matrice N ottenuta "riportando dentro" il controllore K tramite lower LFT ($N = \text{lft}(P, K_{\text{hinfsyn}})$):

$$K_{\text{hinfsyn}} = \min_K \|N(K)\|_\infty$$

```
% Controllore h - infinito
[K_hinfsyn, CL, gamma, info] = hinfsyn(P, 2, 2);

K_hinfsyn_tf = tf(K_hinfsyn);
```

Il comando utilizzato restituisce, oltre al controllore trovato K_{hinfsyn} , il valore *gamma*, che è la norma infinito di N , ossia quella che noi cerchiamo di minimizzare.

Poiché il comando *hinfsyn* restituisce la forma di stato del controllore, è comodo estrarne la matrice di trasferimento così da implementarlo con maggior facilità nel modello *Simulink* del sistema.

5.2 Mu - Analisi

Una volta sintetizzato il controllore, ne è stata fatta un'analisi al fine di verificarne e quantificarne le proprietà di stabilità e performance, sia nominali che robuste, sfruttando come strumento il valore singolare strutturato.

Il controllore è stato quindi riportato dentro lo schema tramite LFT come visto in precedenza:

```
N_hinfsyn = lft(P, K_hinfsyn);

% Risposta in frequenza
N_hinfsyn_frd = frd(N_hinfsyn, omega);
```

ricavando anche la risposta in frequenza del sistema come modello *frd*, necessaria per svolgere le analisi sfruttando il valore singolare strutturato, come mostrato in seguito.

Dopodiché, è stata esplicitata la matrice di incertezza Δ , comprensiva delle incertezze riguardanti sia il sistema fisico che gli attuatori:

$$\Delta = \begin{bmatrix} \Delta_a & 0 \\ 0 & \Delta_G \end{bmatrix}$$

dove Δ_G è la matrice di incertezza riguardante il sistema fisico, ottenuta tramite il comando *lftdata* applicato al modello incerto del sistema G , e Δ_a quella relativa alle incertezze nel modello degli attuatori.

Δ_G risulta essere una matrice diagonale 3x3, costituita da incertezze reali, mentre Δ_a una matrice diagonale complessa 2x2.

```
% Generazione matrice di incertezza totale
```

```
delta_1 = ultidyn('delta_1', [1, 1]);
delta_2 = ultidyn('delta_2', [1, 1]);
Delta = blkdiag(delta_1, delta_2, Delta_G);
```

Nei risultati seguenti sono stati considerati pesi di performance pari a:

$$W_{1i} = \left(\frac{\frac{s}{M^{1/N}} + \omega_B}{s + \omega_B \cdot a^{1/N}} \right)^N$$

per il peso W_1 , con $N = 5$, $a = 0.8$, $\omega_B = 0.1$ e $M = 10^5$

$$W_{2i} = \frac{s}{s + \omega_C}$$

per il peso W_2 , con $\omega_C = 1000$ e $W_{3i} = 0$.

5.2.1 Stabilità nominale

Per verificare la nominale stabilità del sistema, è stata testata la interna stabilità della matrice N . Per far ciò, sono state estratte da N tutte e 16 le sottomatrici che legano i vari ingressi/uscite di N , verificando per ognuna di esse la stabilità tramite comando *isstable* per accertarsi che nessuna presentasse poli instabili.

```
N_int_stab = isstable(N_11)*isstable(N_12)*isstable(N_13)*...*isstable(N_44);
```

```
if (N_int_stab == 0)
    disp('*****')
    disp('Stabilità nominale NON VERIFICATA!!!')
    disp('*****')
else
    disp('*****')
    disp('Stabilità nominale VERIFICATA')
    disp('*****')
end
```

Con la combinazione di pesi scelta, si è verificato che il sistema a ciclo chiuso risulta nominalmente stabile.

5.2.2 Performance nominali

Per verificare le performance nominali, si è usato come indicatore la norma infinito della sottomatrice N_{22} , ossia quella che lega ingressi esogeni e uscite di prestazione del sistema.

$$\|N_{22}\|_{\infty} < 1$$

Idealmente diciamo che la specifica sulle prestazioni nominali è rispettata se questo valore è minore di 1, e in generale desideriamo che esso sia il più basso possibile.

```
hinfnorm_mu_hinfsyn_NP = hinfnorm(N_hinfsyn_22);
```

```
disp ('*****')
disp ('INDICE PRESTAZIONI NOMINALI:')
disp (hinfnorm_mu_hinfsyn_NP)
disp ('*****')
```

Con i pesi scelti, si è ottenuto un valore di questa norma pari a 1.4804. Ciò è dovuto principalmente all'incapacità di rispettare la specifica in bassa frequenza, e si ottengono valori minori qualora si alzi il valore del parametro a del peso di prestazione W_{1i} .

5.2.3 Robusta stabilità

Per testare la robusta stabilità del sistema, si calcola la norma infinito del valore singolare strutturato della matrice $M = N_{11}$, con riferimento all'incertezza strutturata Δ vista precedentemente.

$$\|\mu_{\Delta}(N_{11})\|_{\infty} < 1$$

Tale valore è calcolato numericamente, sfruttando il comando *mussv*

```
% Estraiamo N11
N_hinfsyn_11 = N_hinfsyn_frd(1:length(Delta), 1:length(Delta));

% Struttura dell'incertezza
block_RS_hinfsyn = ones(length(Delta),2);
block_RS_hinfsyn((length(Delta)-length(Delta_G)+1):length(Delta),2) = ...
    zeros(length(Delta_G),1);
block_RS_hinfsyn((length(Delta)-length(Delta_G)+1):length(Delta),1) = ...
    -ones(length(Delta_G),1);

% Calcolo valore singolare strutturato di N11
[bds_RS_hinfsyn, mu_info_RS_hinfsyn] = mussv(N_hinfsyn_11, block_RS_hinfsyn, 'c');

mu_hinfsyn_RS = bds_RS_hinfsyn(:,1);

% Calcolo norma infinito del valore singolare strutturato
hinfnorm_mu_hinfsyn_RS = hinfnorm(mu_hinfsyn_RS);
```

Il sistema risulta essere robustamente stabile qualora questo valore sia minore di 1. Più basso esso è, maggiore sarà l'incertezza che si può sopportare senza rischiare l'instabilità. Successivamente, sono stati calcolati dei margini di stabilità tramite comando *robstab*, riportando la matrice di incertezza all'interno del sistema tramite LFT.

$$F_{\text{hinfsyn}} = \text{lft}(\Delta, N_{\text{hinfsyn}})$$

Maggiori sono questi margini, maggiore è l'incertezza sopportabile.

```
% Calcolo margine di stabilità con comando robstab
```

```
% Riporto dentro l'incertezza tramite LFT
Fu_hinfsyn = lft(Delta, N_hinfsyn);
```

```
[stabmarg_hinfsyn, wcu_hinfsyn_RS] = robstab(Fu_hinfsyn);
```

Con i pesi scelti, il sistema risulta essere robustamente stabile:

```
INDICE ROBUSTA STABILITA:
    0.1421
```

```
*****
MARGINI DI STABILITA:
    LowerBound: 7.0223
    UpperBound: 7.0406
    CriticalFrequency: 1.1190
```

```
*****
```

5.2.4 Robusta performance

Per verificare la robusta performance del sistema, si valuta la norma infinito del valore singolare strutturato della matrice N , con riferimento ad una matrice di incertezza

$$\Delta_{\text{aug}} = \begin{bmatrix} \Delta & 0 \\ 0 & \Delta_P \end{bmatrix} = \begin{bmatrix} \Delta_a & 0 & 0 \\ 0 & \Delta_G & 0 \\ 0 & 0 & \Delta_P \end{bmatrix}$$

dove Δ_P è una matrice di incertezza "fittizia", piena e complessa, di dimensioni pari a quelle di N_{22} .

$$\|\mu_{\Delta_{\text{aug}}}(N)\|_{\infty} < 1$$

```
% Struttura dell'incertezza
block_RP_hinfsyn = ones(length(Delta)+1,2);
block_RP_hinfsyn((length(Delta)-length(Delta_G)+1):length(Delta),2) = ...
    zeros(length(Delta_G),1);
block_RP_hinfsyn((length(Delta)-length(Delta_G)+1):length(Delta),1) = ...
    -ones(length(Delta_G),1);
block_RP_hinfsyn(size(block_RP_hinfsyn,1),:) = ...
    [size(N_hinfsyn_22,1), size(N_hinfsyn_22,2)];

% calcolo del valore singolare strutturato di N
[bds_RP_hinfsyn, mu_info_RP_hinfsyn] = mussv(N_hinfsyn_frd, block_RP_hinfsyn, 'c');

mu_hinfsyn_RP = bds_RP_hinfsyn(:,1);

hinfnorm_mu_hinfsyn_RP = hinfnorm(mu_hinfsyn_RP);
```

Il sistema rispetta la robusta performance qualora questo valore sia inferiore a 1. Analogamente a quanto detto per la robusta stabilità, minore è tale valore, più incertezza si può tollerare senza il rischio di non rispettare le specifiche di performance.

Sono stati inoltre calcolati i margini di performance rispetto ad un certo "valore target" tramite il comando *robgain*. Maggiori sono tali margini, maggiore è l'incertezza sostenibile senza che la norma ∞ del valore singolare strutturato superi tale valore target.

```
% Calcolo dei margini di performance tramite robgain
perf_target_hinfsyn = 2;
[perfmarg_hinfsyn, wcu_hinfsyn_RP] = robgain(Fu_hinfsyn, perf_target_hinfsyn);
```

Con i pesi in esame, si è ottenuto un valore di tale norma infinito all'incirca pari a 1.5, coerentemente al fatto che il controllore non era riuscito a rispettare nemmeno le specifiche di performance nominale.

```
INDICE ROBUSTA PERFORMANCE:
1.4977
```

```
*****
```

```
MARGINI DI PERFORMANCE:
```

```
Valore target:
```

```
2
```

```
LowerBound: 7.3257
```

```
UpperBound: 7.3535
```

```
CriticalFrequency: 1.0896
```

```
*****
```

5.3 Risultati simulazione

Per testare il controllore abbiamo implementato dei modelli *Simulink* del sistema (sia linearizzato che non), e di seguito sono riportati i risultati delle simulazioni.

5.3.1 Sistema linearizzato, senza disturbi

Per questa prova abbiamo applicato il controllore $K_{hinf syn}$ al modello linearizzato senza disturbi. Le condizioni iniziali sono state prese casuali, in un intorno dell'equilibrio. L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 9 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

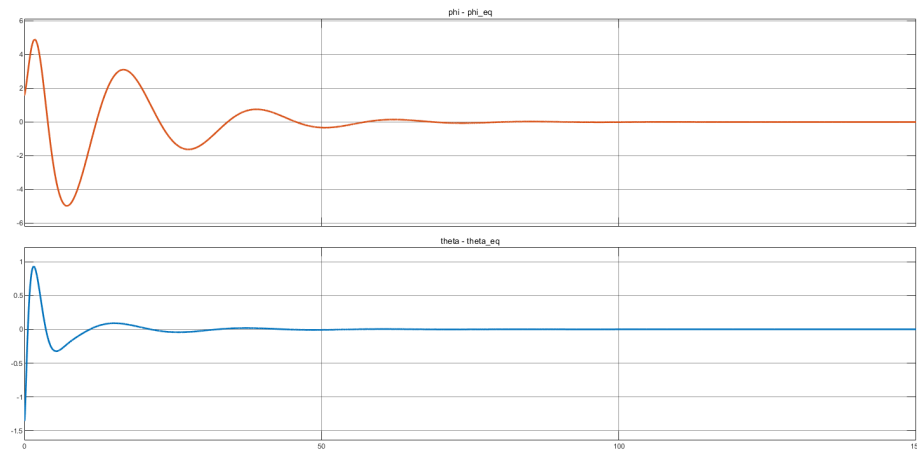


Figura 9: Andamenti delle uscite del sistema linearizzato senza disturbi, controllore hinf syn

Come c'era da aspettarsi, il sistema si dimostra stabile e raggiunge l'equilibrio, pur con un tempo di assestamento non dei migliori (per ϕ è di circa 1 minuto).

5.3.2 Sistema linearizzato, con disturbi

Per questa prova abbiamo applicato il controllore K_{hinfsyn} al modello linearizzato in presenza di disturbi a gradino sull'ingresso al sistema e rumore bianco sulle misure.

Le condizioni iniziali sono state prese casuali, in un intorno dell'equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 10 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

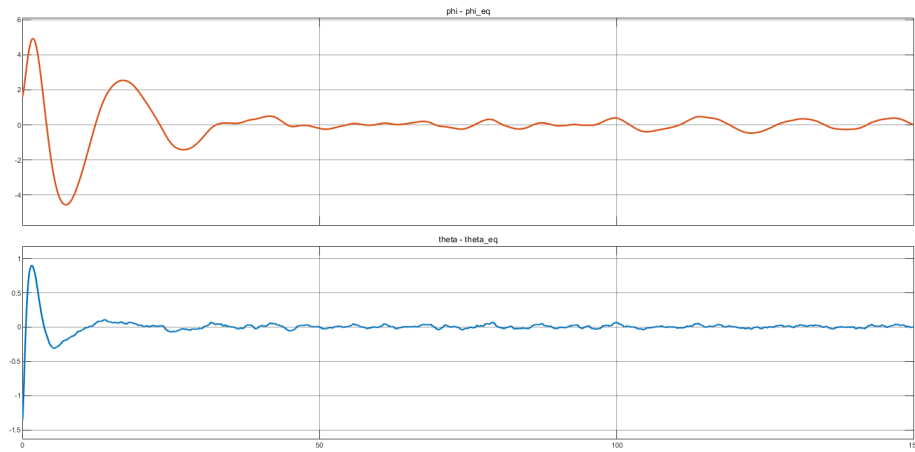


Figura 10: Andamenti delle uscite del sistema linearizzato con disturbi, controllore hinfsyn

Come previsto, il sistema continua a essere stabile anche in presenza di disturbi.

La reiezione dei disturbi è accettabile (il disturbo a gradino sull'ingresso non sposta l'equilibrio del sistema).

5.3.3 Sistema "reale", senza disturbi

Per questa prova abbiamo applicato il controllore $K_{\text{hinf}_{\text{syn}}}$ al modello non lineare senza disturbi. Le condizioni iniziali sono state prese esattamente uguali a quelle di equilibrio. L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento y_{eq}).

In figura 11 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

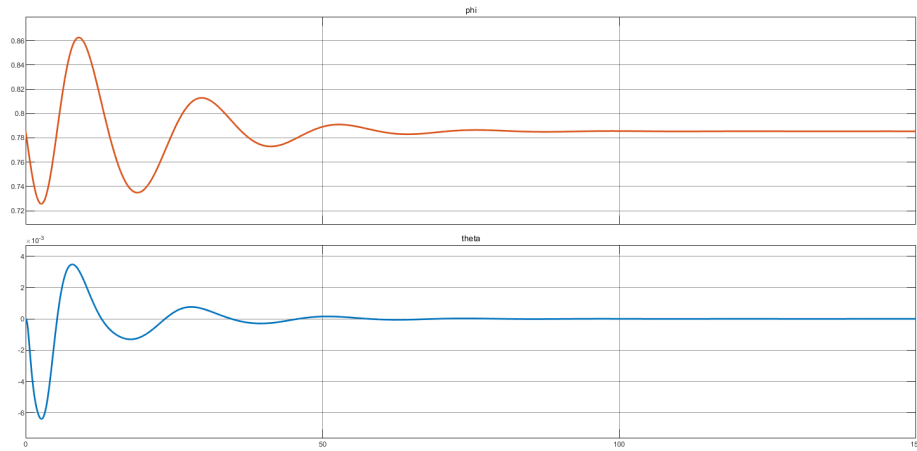


Figura 11: Andamenti delle uscite del sistema non lineare senza disturbi, controllore hinf_{syn}

Il sistema risulta stabile in questo intorno delle condizioni di equilibrio, e il tracking è analogo al caso del linearizzato.

5.3.4 Sistema "reale", con disturbi

Per questa prova abbiamo applicato il controllore $K_{\text{hinf}_{\text{syn}}}$ al modello non lineare in presenza di disturbi a gradino sull'ingresso al sistema e rumore bianco sulle misure.

Le condizioni iniziali sono state prese esattamente uguali a quelle di equilibrio.

L'obiettivo della simulazione è mantenere il sistema in posizione di equilibrio (riferimento y_{eq}).

In figura 12 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

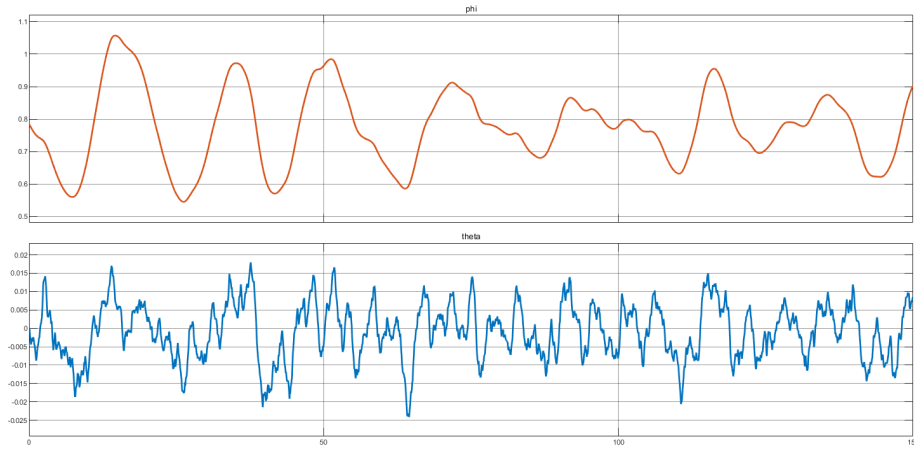


Figura 12: Andamenti delle uscite del sistema linearizzato con disturbi, controllore hinf_{syn}

Il sistema conserva la stabilità anche con i disturbi applicati, ma le performance in questo caso sono pessime, sia per quanto riguarda il tracking che la reiezione dei disturbi.

6 Controllore D-K

Questo controllore è stato sintetizzato tramite l'algoritmo D-K iteration utilizzando il comando *musyn*.

L'algoritmo prevede i seguenti passi:

1. Inizializzare la matrice $D(s) = I_d$ con dimensioni pari a quelle della matrice P (questa versione dell'algoritmo è applicabile solo se P è quadrata)
2. Trovare un controllore $K_{DK}(s)$ che minimizza $\|lft(DPD^{-1}, K_{DK})\|_\infty$
3. Trovare $D(s)$ che minimizza $\|Dlft(P, K_{DK})D^{-1}\|_\infty$
4. Ripetere i punti (2) e (3) fino a che non si ottiene un controllore con prestazioni soddisfacenti o si arriva a convergenza dell'algoritmo.

Il controllore così ottenuto è sub-ottimo² nel senso della robusta performance, ovvero:

$$K_{DK}(s) = \min_{K(s)} \|\mu_{\Delta_{aug}}(lft(P(s), K(s)))\|_\infty$$

6.1 Sintesi

Per la sintesi è necessario utilizzare la Δ del sistema che prendiamo dalla forma di Doyle.

```
% Generazione matrice di incertezza totale

delta_1 = ultidyn('delta_1', [1, 1]);
delta_2 = ultidyn('delta_2', [1, 1]);
Delta = blkdiag(delta_1, delta_2, Delta_G);
```

Dopodiché è necessario definire una matrice $E = lft(\Delta, P)$ dove la matrice P è sempre quella della forma di Doyle. Questa matrice E viene poi utilizzata dal comando *musyn* per eseguire la D-K iteration

```
% Sintesi controllore D-K usando funzione musyn
[K_DK, CLperf_DK] = musyn(E, 2, 2);

% Funzione di trasferimento del controllore
K_DK_tf = tf(K_DK);
```

Come per il controllore $\mathcal{H}_\infty$ convertiamo il controllore in funzione di trasferimento per applicarlo al modello Simulink del sistema.

Si riportano di seguito i risultati degli step della procedura

²Si intende che è ottimo nella classe di controllori sintetizzabili con questa tecnica. Ciò non esclude che possa esistere un controllore migliore nel senso RP.

D-K ITERATION SUMMARY:

Robust performance				Fit order
Iter	K Step	Peak MU	D Fit	D
1	1.639	1.499	1.514	24
2	1.03	0.7403	0.745	20
3	0.3799	0.2742	0.2775	34
4	0.1716	0.1696	0.1711	36
5	0.1643	0.1637	0.1652	42
6	0.1642	0.1635	0.1649	44
7	0.1641	0.1635	0.1646	40

Best achieved robust performance: 0.163

Risulta evidente già da subito che le prestazioni sono migliori del controllore $\mathcal{H}_\infty$, che si nota essere il primo passo della procedura.

6.2 Mu - Analisi

Una volta sintetizzato il controllore, ne è stata fatta un'analisi al fine di verificarne e quantificarne le proprietà di stabilità e performance, sia nominali che robuste, sfruttando come strumento il valore singolare strutturato.

Il controllore è stato quindi riportato dentro lo schema tramite LFT come visto in precedenza:

```
N_DK = lft(P,K_DK);
```

```
% Risposta in frequenza  
N_DK_frd = frd(N_DK, omega);
```

ricavando anche la risposta in frequenza del sistema come modello *frd*, necessaria per svolgere le analisi sfruttando il valore singolare strutturato, come mostrato in seguito.

Nei risultati seguenti sono stati considerati pesi di performance pari a:

$$W_{1i} = \left(\frac{\frac{s}{M^{1/N}} + \omega_B}{s + \omega_B \cdot a^{1/N}} \right)^N$$

per il peso W_1 , con $N = 5$, $a = 0.8$, $\omega_B = 0.1$ e $M = 10^5$

$$W_{2i} = \frac{s}{s + \omega_C}$$

per il peso W_2 , con $\omega_C = 1000$ e $W_{3i} = 0$.

6.2.1 Stabilità nominale

Per verificare la nominale stabilità del sistema, è stata testata la interna stabilità della matrice N . Per far ciò, sono state estratte da N tutte e 16 le sottomatrici che legano i vari ingressi/uscite di N , verificando per ognuna di esse la stabilità tramite comando *isstable* per accertarsi che nessuna presentasse poli instabili.

```
N_int_stab = isstable(N_11)*isstable(N_12)*isstable(N_13)*...*isstable(N_44);
```

```
if (N_int_stab == 0)  
    disp('*****')  
    disp('Stabilità nominale NON VERIFICATA!!!')  
    disp('*****')  
else  
    disp('*****')  
    disp('Stabilità nominale VERIFICATA')  
    disp('*****')  
end
```

Con la combinazione di pesi scelta, si è verificato che il sistema a ciclo chiuso risulta nominalmente stabile.

6.2.2 Performance nominali

Per verificare le performance nominali, si è usato come indicatore la norma infinito della sottomatrice N_{22} , ossia quella che lega ingressi esogeni e uscite di prestazione del sistema.

$$\|N_{22}\|_{\infty} < 1$$

Idealmente la specifica sulle prestazioni nominali è rispettata se questo valore è minore di 1, e in generale desideriamo che sia il più basso possibile.

```
hinfnorm_mu_DK_NP = hinfnorm(N_DK_22);
```

Il valore ottenuto è 0.1625, il sistema nominale rispetta le specifiche con ottimi risultati.

6.2.3 Robusta stabilità

Per testare la robusta stabilità del sistema, si calcola la norma infinito del valore singolare strutturato della matrice $M = N_{11}$, con riferimento all'incertezza strutturata Δ vista precedentemente.

$$\|\mu_{\Delta}(N_{11})\|_{\infty} < 1$$

Tale valore è calcolato numericamente, sfruttando il comando *mussv*

```
% Estraiamo N11
N_DK_11 = N_DK_frd(1:length(Delta), 1:length(Delta));

% Struttura dell'incertezza
block_RS_DK = ones(length(Delta),2);
block_RS_DK((length(Delta)-length(Delta_G)+1):length(Delta),2) = ...
    zeros(length(Delta_G),1);
block_RS_DK((length(Delta)-length(Delta_G)+1):length(Delta),1) = ...
    -ones(length(Delta_G),1);

% Calcolo valore singolare strutturato di N11
[bds_RS_DK, mu_info_RS_DK] = mussv(N_DK_11, block_RS_DK, 'c');

mu_DK_RS = bds_RS_DK(:,1);

% Calcolo norma infinito del valore singolare strutturato
hinfnorm_mu_DK_RS = hinfnorm(mu_DK_RS);
```

Il sistema risulta essere robustamente stabile qualora questo valore sia minore di 1. Più basso esso è, maggiore sarà l'incertezza che si può sopportare senza rischiare l'instabilità.

Successivamente, Sono stati calcolati dei margini di stabilità tramite comando *robstab*, riportando la matrice di incertezza all'interno del sistema.

$$F_{DK} = lft(\Delta, N_{DK})$$

Maggiori sono questi margini, maggiore è l'incertezza sopportabile.

```
% Calcolo margine di stabilità con comando robstab
```

```
% Riporto dentro l'incertezza tramite LFT
Fu_DK = lft(Delta, N_DK);
```

```
[stabmarg_DK, wcu_DK_RS] = robstab(Fu_DK);
```

Il sistema risulta essere robustamente stabile:

```
INDICE ROBUSTA STABILITA:
```

```
0.1421
```

```
*****
```

```
MARGINI DI STABILITA:
```

```
LowerBound: 7.0252
```

```
UpperBound: 7.0392
```

```
CriticalFrequency: 1.1103
```

```
*****
```

6.2.4 Robusta performance

Per verificare la robusta performance del sistema, si valuta la norma infinito del valore singolare strutturato della matrice N , con riferimento ad una matrice di incertezza

$$\Delta_{\text{aug}} = \begin{bmatrix} \Delta & 0 \\ 0 & \Delta_P \end{bmatrix} = \begin{bmatrix} \Delta_a & 0 & 0 \\ 0 & \Delta_G & 0 \\ 0 & 0 & \Delta_P \end{bmatrix}$$

dove Δ_P è una matrice di incertezza "fittizia", piena e complessa, di dimensioni pari a quelle di N_{22} .

$$\|\mu_{\Delta_{\text{aug}}}(N)\|_{\infty} < 1$$

```
% Struttura dell'incertezza
```

```
block_RP_DK = ones(length(Delta)+1,2);
```

```
block_RP_DK((length(Delta)-length(Delta_G)+1):length(Delta),2) = ...
```

```
zeros(length(Delta_G),1);
```

```
block_RP_DK((length(Delta)-length(Delta_G)+1):length(Delta),1) = ...
```

```
-ones(length(Delta_G),1);
```

```
block_RP_DK(size(block_RP_hinfsyn,1),:) = ...
```

```
[size(N_hinfsyn_22,1), size(N_hinfsyn_22,2)];
```

```
% calcolo del valore singolare strutturato di N
```

```
[bds_RP_DK, mu_info_RP_DK] = mussv(N_DK_frd, block_RP_DK, 'c');
```

```
mu_DK_RP = bds_RP_DK(:,1);
```

```
hinfnorm_mu_hinfsyn_RP = hinfnorm(mu_hinfsyn_RP);
```

Il sistema rispetta la robusta performance qualora questo valore sia inferiore a 1. Analogamente a quanto detto per la robusta stabilità, minore è tale valore, più incertezza si può tollerare senza il

rischio di non rispettare le specifiche di performance.

Sono stati inoltre calcolati i margini di performance rispetto ad un "valore target" tramite il comando *robgain*. Maggiori sono tali margini, maggiore è l'incertezza sostenibile senza che la norma ∞ del valore singolare strutturato superi tale valore target.

```
% Calcolo dei margini di performance tramite robgain
perf_target_DK = 2;
[perfmarg_DK, wcu_DK_RP] = robgain(Fu_DK, perf_target_DK);
```

Si può notare che la proprietà di robusta performance è verificata.

```
      INDICE ROBUSTA PERFORMANCE:
      0.1633
*****
MARGINI DI PERFORMANCE:
Valore target:
      2

      LowerBound: 6.9994
      UpperBound: 7.0156
CriticalFrequency: 1.1127
*****
```

6.3 Risultati simulazione

Per testare il controllore abbiamo implementato dei modelli Simulink del sistema (sia linearizzato che non), e di seguito riporteremo i risultati delle simulazioni.

6.3.1 Sistema linearizzato, senza disturbi

Per questa prova abbiamo applicato il controllore K_{DK} al modello linearizzato senza disturbi.

Le condizioni iniziali sono state prese casuali, in un intorno dell'equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 13 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

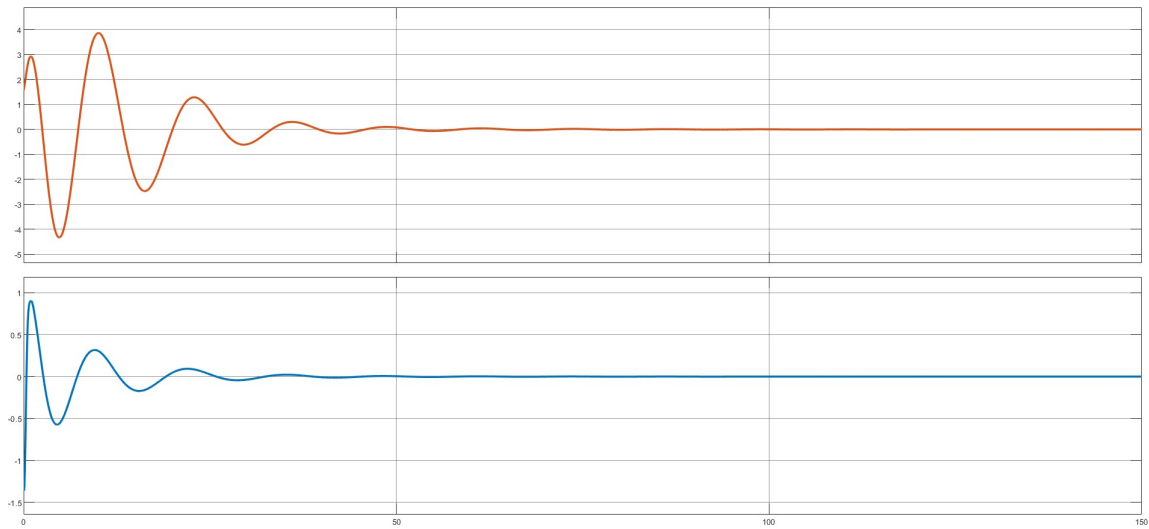


Figura 13: Andamenti delle uscite del sistema linearizzato senza disturbi, controllore DK

Come c'era da aspettarsi il sistema è stabile, e la prestazione sul tracking è analoga a quella del controllore K_{hinfyn} .

6.3.2 Sistema linearizzato, con disturbi

Per questa prova abbiamo applicato il controllore K_{DK} al modello linearizzato in presenza di disturbi a gradino sull'ingresso al sistema e rumore bianco sulle misure.

Le condizioni iniziali sono state prese casuali, in un intorno dell'equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 14 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

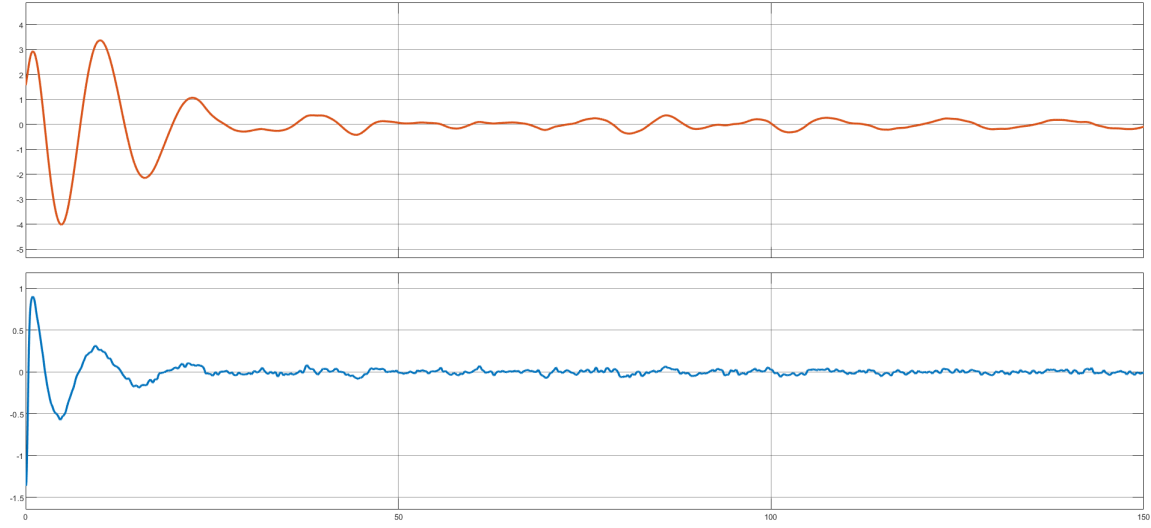


Figura 14: Andamenti delle uscite del sistema linearizzato con disturbi, controllore DK

Come previsto, il sistema continua a essere stabile anche in presenza di disturbi.

La reiezione dei disturbi è buona (il disturbo a gradino sull'ingresso non sposta l'equilibrio del sistema).

6.3.3 Sistema "reale", senza disturbi

Per questa prova abbiamo applicato il controllore K_{DK} al modello non lineare senza disturbi.

Le condizioni iniziali sono state prese esattamente uguali a quelle di equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento y_{eq}).

In figura 15 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

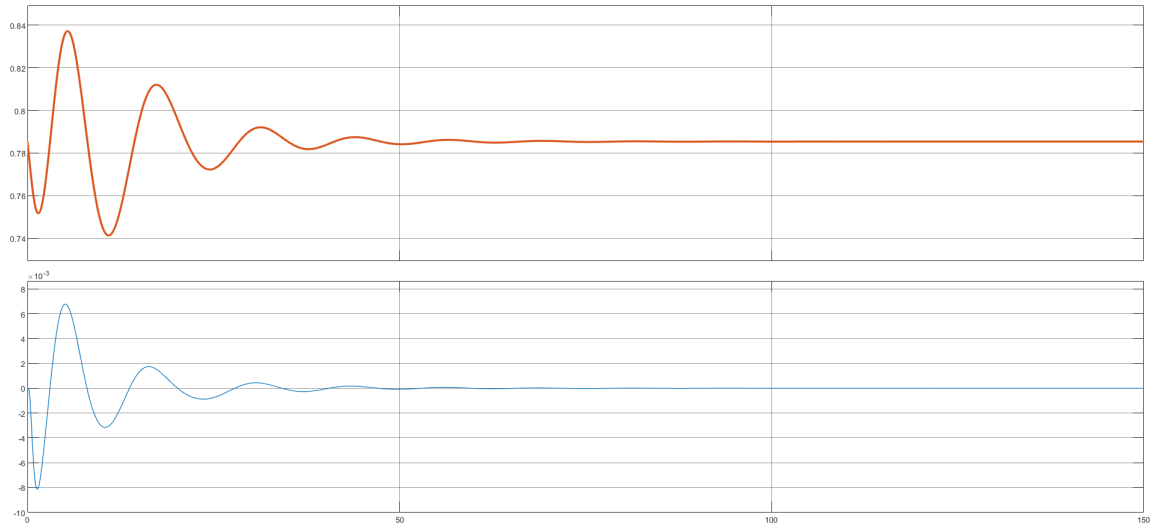


Figura 15: Andamenti delle uscite del sistema non lineare senza disturbi, controllore DK

Il sistema risulta stabile in questo intorno delle condizioni di equilibrio. Per quanto riguarda le prestazioni ha senso confrontarle con i risultati di $K_{hinfsyn}$, mostrati in figura 11:

- Il tempo di assestamento per ϕ è diminuito di circa 10 s, quindi un miglioramento di circa il 15%.
- La massima sovralongazione su ϕ è diminuita, siamo passati da un picco di sovralongazione del 9.5% al 6.6%.
- La sovralongazione su θ di contro è aumentata, tuttavia il picco massimo è di 0.008 rad, quindi molto basso.

6.3.4 Sistema "reale", con disturbi

Per questa prova abbiamo applicato il controllore K_{DK} al modello non lineare in presenza di disturbi a gradino sull'ingresso al sistema e rumore bianco sulle misure.

Le condizioni iniziali sono state prese esattamente uguali a quelle di equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento y_{eq}).

In figura 16 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

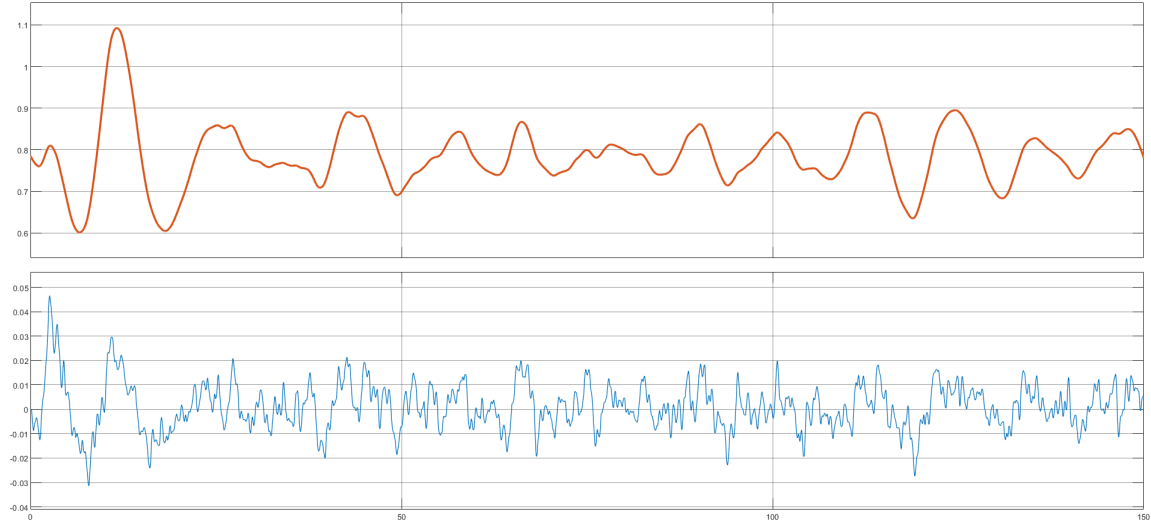


Figura 16: Andamenti delle uscite del sistema linearizzato con disturbi, controllore DK

Il sistema conserva la stabilità anche con i disturbi applicati. Per quanto riguarda la reiezione dei disturbi ha senso confrontarla con i risultati di $K_{hinf_{syn}}$, mostrati in figura 12:

- L'ampiezza dell'oscillazione a regime su ϕ è diminuita da circa 0.16 *rad* a circa 0.10 *rad*.
- L'oscillazione su θ è rimasta pressochè invariata.

7 Controllore LQG

Infine, è stato sintetizzato un controllore tramite la tecnica LQG, che mette insieme il Regolatore Lineare Quadratico (LQR) e la stima dello stato mediante filtro di Kalman.

Il controllore LQR è un controllore *state-feedback*, in cui il segnale di controllo è della forma

$$u = -K_c x$$

che alloca i poli del sistema a ciclo chiuso in modo ottimo per minimizzare l'indice di costo

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

con

- $Q = Q^T \geq 0$
- $R = R^T > 0$

Modificando le matrici di peso Q e R , è possibile decidere se dare maggior importanza alla vicinanza tra lo stato del sistema e lo stato desiderato³, oppure ad avere un'energia di controllo bassa.

Nel controllo LQG la differenza sta nel fatto che, invece di retroazionare lo stato, si retroaziona la sua stima fornita dal filtro di Kalman, il quale prende come ingressi le uscite dei sensori (y) e gli ingressi al sistema (u) e restituisce la stima dello stato ($\hat{x}$).⁴

In questo caso si è deciso di mettere insieme le matrici di trasferimento del sistema e degli attuatori, scrivendo la matrice di trasferimento complessiva

$$G_{\text{tot}} = G \cdot G_a$$

e considerando uno stato a sei componenti che include gli stati legati agli attuatori.

A differenza di quanto fatto per gli altri controllori, in questo caso è stato supposto che le uscite del sistema fossero quelle dei sensori usati in precedenza per il progetto di Identificazione dei Sistemi Incerti, ossia:

$$\begin{cases} y_1 = -g \cdot \cos \theta \\ y_2 = l \cdot \sin \phi \end{cases}$$

Queste espressioni sono state linearizzate nell'intorno dello stato di equilibrio desiderato, in modo tale da scrivere la matrice

$$C = \begin{bmatrix} 0 & g & 0 & 0 & 0 & 0 \\ \frac{l}{\sqrt{2}} & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

³Nel nostro caso, lavorando sul sistema linearizzato, lo stato desiderato è un vettore di zeri

⁴Per la sintesi di questo controllore si è utilizzato il filtro di Kalman con guadagno costante

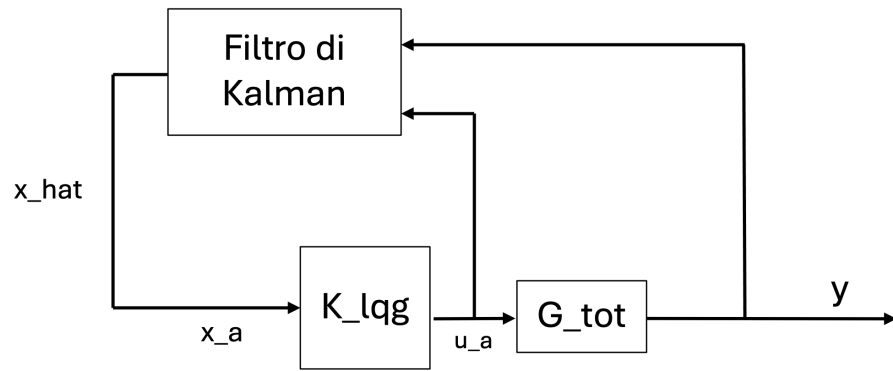


Figura 17: Schema a blocchi del controllore LQG

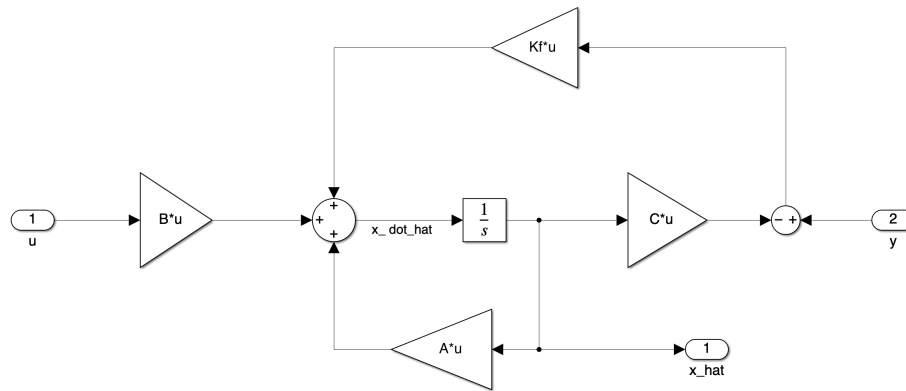


Figura 18: Schema del filtro di Kalman

7.1 Sintesi

La sintesi di questo controllore fondamentalmente si riduce quindi alla ricerca dei guadagni ottimi K_c e K_f relativi al controllore LQR e al filtro di Kalman. Essi sono stati ottenuti risolvendo due Equazioni Algebriche di Riccati (ARE) tramite il comando *icare*.

Si noti che le matrici A, B, C, D utilizzate in questo caso sono quelle relative al sistema complessivo G_{tot} .

- Per il controllore si cerca il guadagno ottimo

$$K_c = R^{-1} B^T P_c$$

tale che P_c sia l'unica soluzione simmetrica e semi-definita positiva della ARE:

$$A^T P_c + P_c A - P_c B R^{-1} B^T P_c + M^T Q M = 0$$

```
% ARE per trovare Kc (controllore)
% Il controllore Kc trovato con questa procedura è tale per cui la nuova
% matrice dinamica a ciclo chiuso del sistema sia (A - B*Kc).
```

```
[Pc, Kc, eig_CL] = icare(A, B, Q, R, zeros(6,2), eye(6), zeros(6,6));
```

- Per il filtro, si cerca il guadagno

$$K_f = P_f C^T V^{-1}$$

tale che P_f sia l'unica soluzione simmetrica e semi-definita positiva della ARE:

$$P_f A^T + A P_f - P_f C^T V^{-1} C P_f + W = 0$$

```
% ARE per trovare Kf (matrice dei guadagni del filtro di Kalman)
% La matrice dei guadagni Kf è tale per cui la matrice dinamica
% dello stimatore sia (A - Kf*C)
```

```
[Pf, Kf, eig_kalman] = icare(A', C', W, V, zeros(6,2), ...
    eye(6), zeros(6,6));
```

```
% La ARE restituisce Kf e Pf trasposti rispetto a quelli che ci servono
Pf = Pf';
Kf = Kf';
```

dove W e V sono le matrici di covarianza dei rumori di processo e di misura, i cui valori sono stati presi in accordo a quelli utilizzati durante il progetto di ISI (essendo matrici di covarianza, sono necessariamente definite positive, entrambe).

```
% Matrice di covarianza rumore di processo
W = blkdiag(0.2, 0.2, 0.1, 0.1, 0.15, 0.15);
```

```
% Matrice di covarianza rumore di misura (Varianze dei sensori usati nel  
% progetto di ISI)  
V = blkdiag(0.003^2, 0.02^2);
```

7.2 LQG con integratore

È stata realizzata inoltre una variante del controllore LQG che presentasse anche azione integrale. In questo caso, oltre alla stima dello stato fornita dal filtro di Kalman, si retroaziona anche l'integrale della differenza tra l'uscita del sistema e l'uscita desiderata.

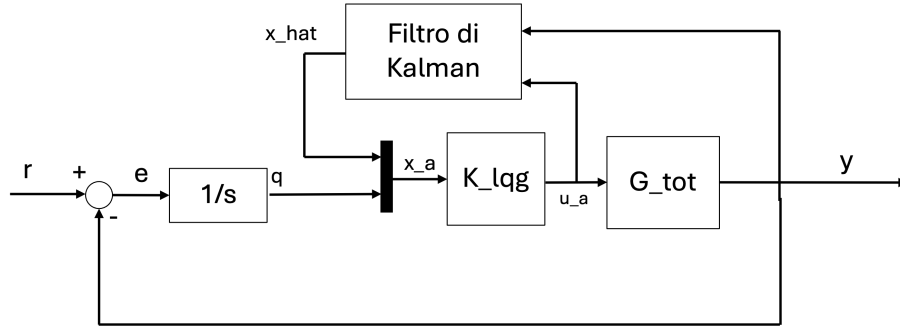


Figura 19: Schema del controllore LQG con azione integrale

Per fare ciò, si aumenta lo stato del sistema aggiungendo una variabile q tale che

$$\dot{q} = r - Cx$$

e si scrivono le matrici aumentate

$$A_{\text{aug}} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \quad B_{\text{aug}} = \begin{bmatrix} B \\ 0 \end{bmatrix}$$

Il guadagno ottimo del filtro risulta essere lo stesso del controllore LQG senza integratore, mentre per trovare il nuovo guadagno ottimo del controllore si risolve nuovamente una ARE, avendo cura di modificare la matrice Q per pesare adeguatamente la componente integrale

```
% Risolvo la ARE per il controllore LQG con azione integrale
[Pc_i, Kc_i, eig_CL_i] = icare(Aa, Ba, Qa, R, zeros(8,2), eye(8), zeros(8,8));
```

7.3 Risultati simulazione

7.3.1 Sistema linearizzato, senza disturbi

Per questa prova abbiamo applicato il controllore K_{LQG} al modello linearizzato senza disturbi. Le condizioni iniziali sono state prese random in un intorno dell'equilibrio. L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 20 sono riportati gli andamenti della configurazione nelle condizioni descritte sopra.

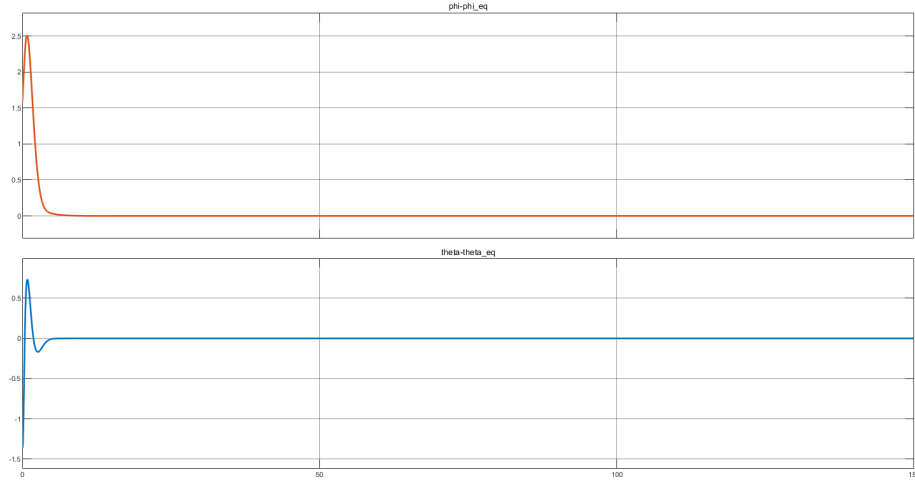


Figura 20: Andamenti della configurazione simulazione sistema linearizzato senza disturbi, controllore LQG

Il sistema risulta stabile. Le performance di tracking in questo caso (sul sistema linearizzato) sono nettamente migliori rispetto ai controllori K_{hinfsyn} e K_{DK} , infatti:

- Il tempo di assestamento è di circa 5 s, rispetto a circa 1 min degli altri controllori.
- Su ϕ è presente un'iniziale inversione, ma non c'è sovraelongazione.

7.3.2 Sistema linearizzato, con disturbi

Per questa prova abbiamo applicato il controllore K_{LQG} al modello linearizzato con disturbi sia di processo che di misura.

I disturbi sono entrambi realizzazioni di rumori bianchi con matrici di covarianza rispettivamente W e V , le stesse usate per la sintesi della matrice dei guadagni del filtro di Kalman.

Le condizioni iniziali sono state prese random in un intorno dell'equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 21 sono riportati gli andamenti della configurazione nelle condizioni descritte sopra.

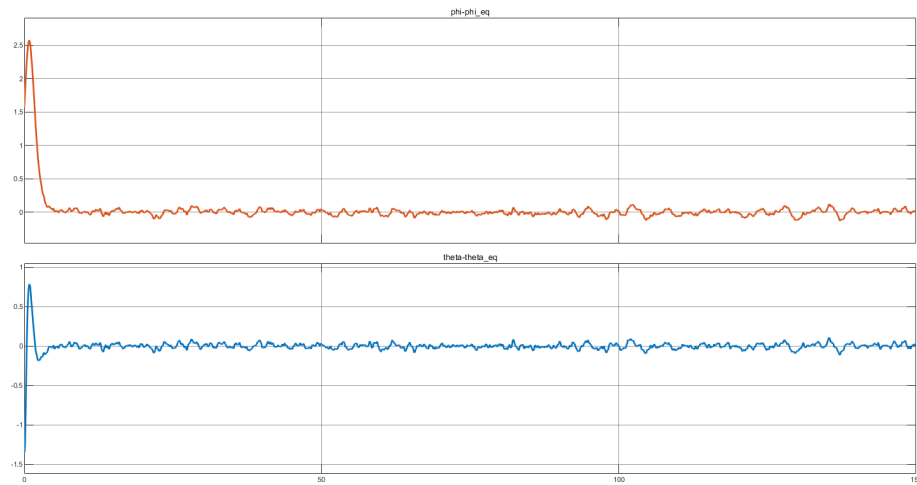


Figura 21: Andamenti della configurazione simulazione sistema linearizzato con disturbi, controllore LQG

Notiamo che la presenza dei disturbi implementati come rumore bianco non ha alcun effetto sulla stabilità o sul tracking. L'unico effetto che hanno è introdurre una componente di rumore in alta frequenza sul comportamento del sistema.

7.3.3 Sistema linearizzato, con disturbo a gradino su ingresso

Per questa prova abbiamo applicato il controllore K_{LQG} al modello linearizzato in presenza di un disturbo a gradino sull'ingresso al sistema.

Le condizioni iniziali sono state prese random in un intorno dell'equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 22 sono riportati gli andamenti della configurazione nelle condizioni descritte sopra.

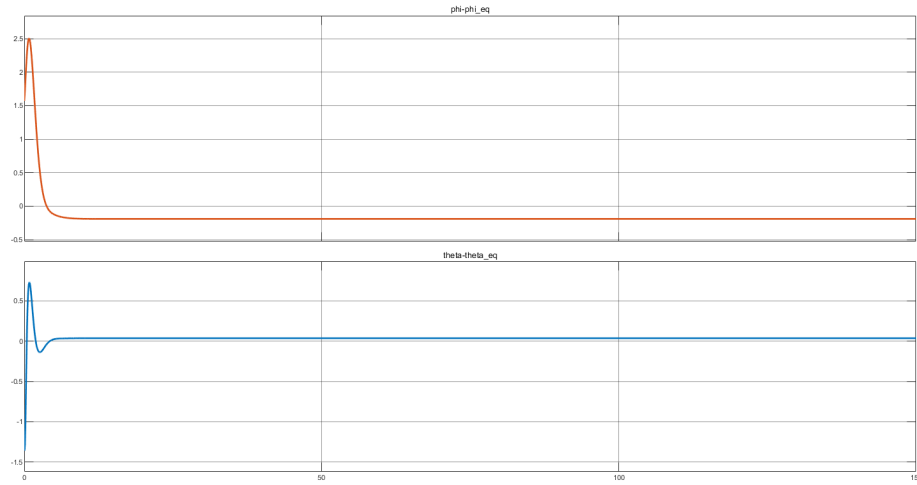


Figura 22: Andamenti della configurazione simulazione sistema linearizzato con disturbo a gradino su ingresso, controllore LQG

Notiamo che la presenza del disturbo a gradino sull'ingresso introduce un errore a regime che non viene reiettato da questo controllore, il che è naturale in quanto non presenta un'azione integrale.

7.3.4 Sistema linearizzato, con disturbo a gradino su ingresso, con azione integrale di controllo

Per questa prova abbiamo applicato il controllore K_{LQG} con azione integrale al modello linearizzato in presenza di un disturbo a gradino sull'ingresso al sistema.

Le condizioni iniziali sono state prese random in un intorno dell'equilibrio.

L'obiettivo della simulazione è portare e mantenere il sistema in posizione di equilibrio (riferimento nullo).

In figura 23 sono riportati gli andamenti della configurazione nelle condizioni descritte sopra.

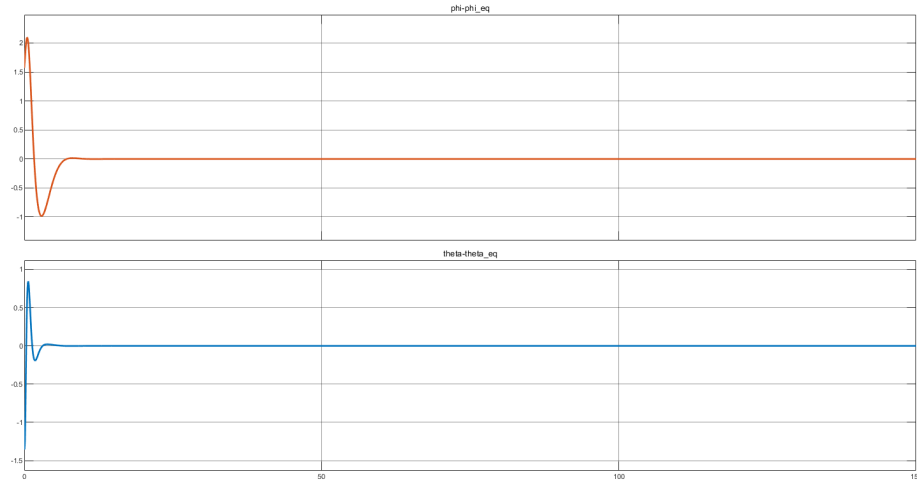


Figura 23: Andamenti della configurazione simulazione sistema linearizzato con disturbo a gradino su ingresso, controllore LQG con azione integrale

Come c'era da aspettarsi, l'introduzione dell'azione integrale ha permesso di reiettare completamente l'errore a regime introdotto dal disturbo a gradino sull'ingresso.

7.3.5 Sistema "reale", senza disturbi

Per questa prova abbiamo applicato il controllore K_{LQG} al modello non lineare senza disturbi.

Le condizioni iniziali sono state prese esattamente uguali a quelle di equilibrio.

L'obiettivo della simulazione è mantenere il sistema in posizione di equilibrio.

In figura 24 sono riportati gli andamenti delle uscite nelle condizioni descritte sopra.

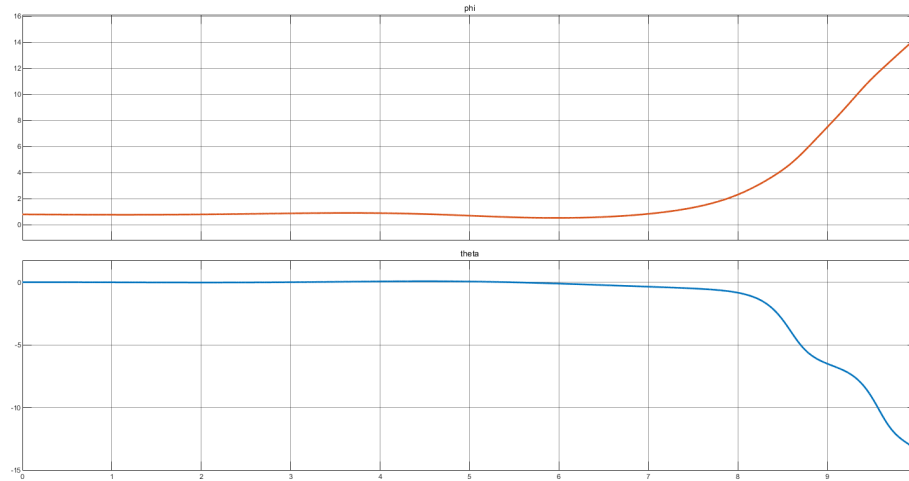


Figura 24: Andamenti della configurazione simulazione sistema non lineare senza disturbi, controllore LQG

Il sistema risulta instabile. L'andamento che teoricamente avremmo dovuto ottenere è:

- $\phi = \frac{\pi}{4} \text{ rad}$ costante
- $\theta = 0 \text{ rad}$ costante

Questo comportamento potrebbe essere causato da derive numeriche in fase di simulazione, che portano il segnale di errore ad essere diverso da 0.

8 Extra

8.1 Calcolo manuale della matrice N_G

Si è anche cercato di ricavare manualmente l'espressione della matrice N_G , al fine di verificare che combaciassse con quella fornita dal comando *lftdata*. Costruendo a partire dalla funzione di trasferimento $G(s)$ il corrispondente schema a blocchi, abbiamo ricavato a partire da questo la N_G , estraendo i blocchi δ relativi alle incertezze.

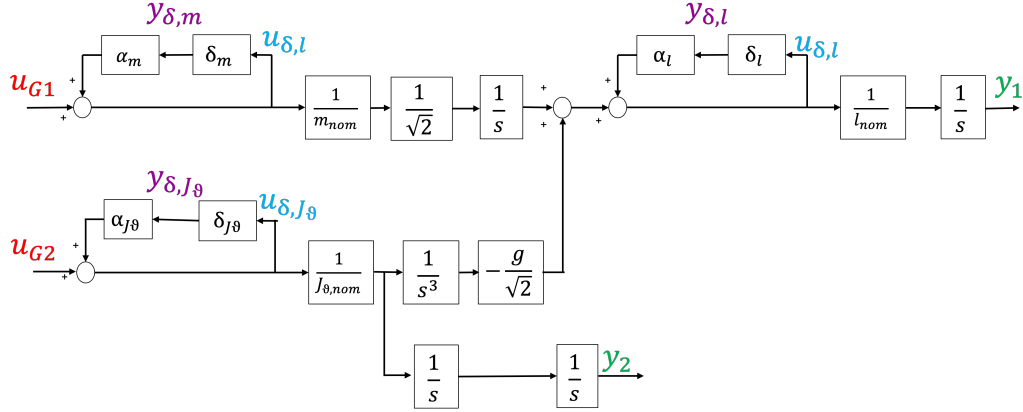


Figura 25: Schema a blocchi di G con relative incertezze

$$N_G = \begin{bmatrix} \alpha_{j\theta} & 0 & 0 & 0 & 1 \\ -g\alpha_{j\theta} & \alpha_l & \frac{\alpha_m}{m_{nom}\sqrt{2}s} & \frac{1}{m_{nom}\sqrt{2}s} & \frac{-g}{J_{\theta nom}\sqrt{2}s^3} \\ \frac{J_{\theta nom}\sqrt{2}s^3}{0} & 0 & \alpha_m & 1 & 0 \\ -g\alpha_{j\theta} & \alpha_l & \alpha_m & 1 & -g \\ \frac{J_{\theta nom}l_{nom}\sqrt{2}s^4}{J_{\theta nom}l_{nom}\sqrt{2}s^4} & \frac{l_{nom}s}{l_{nom}s} & \frac{m_{nom}l_{nom}\sqrt{2}s^2}{m_{nom}l_{nom}\sqrt{2}s^2} & \frac{m_{nom}l_{nom}\sqrt{2}s^2}{m_{nom}l_{nom}\sqrt{2}s^2} & \frac{J_{\theta nom}l_{nom}\sqrt{2}s^4}{J_{\theta nom}l_{nom}\sqrt{2}s^4} \\ \frac{\alpha_{j\theta}}{J_{\theta nom}s^2} & 0 & 0 & 0 & \frac{1}{J_{\theta nom}s^2} \end{bmatrix}$$

Figura 26: N_G ricavata dallo schema a blocchi

Si è osservato tuttavia che la N_G trovata con questo metodo non corrisponde perfettamente a quella calcolata da MatLabTM. In particolare, si può notare che le ultime righe della matrice, quelle relative alle uscite del sistema, risultavano congruenti con quelle fornite dal software, mentre le prime, relative agli ingressi dei blocchi δ , apparivano diverse. Si ipotizza che tale differenza sia dovuta ad un diverso modo di rappresentare lo schema a blocchi interno a MatLabTM. Si ritiene perciò che entrambe le espressioni di N_G possano considerarsi valide.

8.2 Incertezze parametriche sugli attuatori

Abbiamo provato a generare un modello di incertezza parametrico anche per gli attuatori, estraendo le matrici Δ_{G_a} e N_{G_a} dal sistema incerto relativo agli attuatori G_a tramite il comando *lftdata*, così come fatto in precedenza per il sistema fisico.

Anche in questo caso, abbiamo cercato di ricavare la matrice N_{G_a} manualmente, partendo dal relativo schema a blocchi, notando nuovamente delle leggere discrepanze con quella fornita dal comando. Per gli stessi motivi esposti nella sezione 8.1, entrambe le espressioni sono state considerate valide.

Ad ogni modo, nelle parti successive del progetto si è preferito utilizzare il modello con incertezza concentrata presentato nella sezione 3.2.

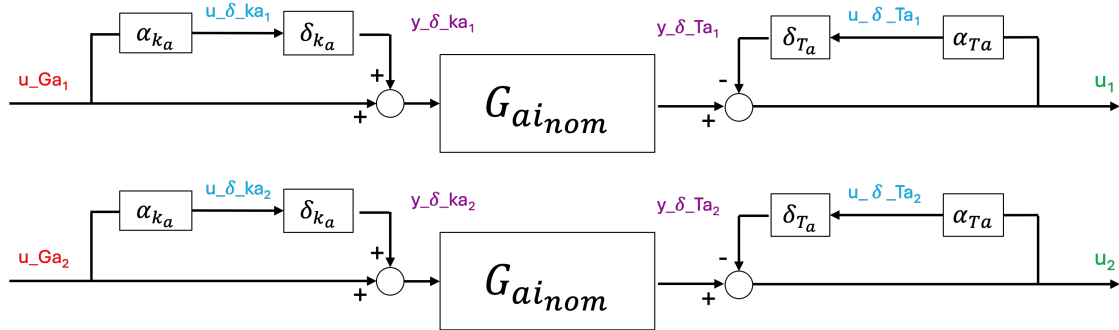


Figura 27: Schema a blocchi di G_a comprensivo di incertezza

8.3 Sintesi controllore D-K con implementazione manuale dell'algoritmo

Abbiamo sintetizzato un controllore implementando manualmente l'algoritmo D-K iteration senza utilizzare il comando *musyn*, che riportiamo di seguito:

```
mu_diff = 100000; % Generico valore elevato per far partire algoritmo
mu_step_behind = mu_diff;
max_steps = 5; % Numero di passi da far compiere all'algoritmo
j = 1;

D = tf(eye(length(P)));

mu_series = zeros(1,max_steps); % Per salvare i mu ai vari step

K_DK_minimizing_mu = ss(tf(eye(2))); % Per salvare l'attuale controllore migliore
step_minimizing_mu = 1; % Per salvare lo step al quale abbiamo ottenuto l'attuale controllore migliore

while j<=max_steps

    [K_DK_manual, CL_manual, gam] = hinfsyn(D*P*inv(D), 2, 2);

    N_DK_manual = frd(lft(D*P*inv(D), K_DK_manual),omega);

    [mu_bnds, info] = mussv(N_DK_manual, block_RP_DK);

    mu = hinfnorm(mu_bnds(:,1));

    disp('*****')
    disp('STEP : ')
    disp(j)
    disp('mu:')
    disp(mu)
    mu_series(j) = mu;

    if mu <= min(mu_series(1:j))
        K_DK_minimizing_mu = K_DK_manual;
        step_minimizing_mu = j;
    end

    [dsysl,dsysr] = mussvunwrap(info);
    dsysl = dsysl/dsysl(6,6);
    d1 = fitfrd(genphase(dsysl(1,1)),4);
    d2 = d1;
    d3 = d1;
    d4 = d1;
    d5 = d1;
    d6 = tf(1);
```

```

d7 = tf(1);
d8 = tf(1);
d9 = tf(1);
d10 = tf(1);
d11 = tf(1);

D_new = blkdiag(d1, d2, d3, d4, d5, d6, d7, d8, d9, d10, d11, tf(1), tf(1));
D = D_new*D;

mu_diff = mu_step_behind - mu;
mu_step_behind = mu;

disp ('mu_diff:')
disp (mu_diff)
disp ('*****')

j = j+1;
end

```

La procedura dà i seguenti risultati:

```

>> CSI_controllore_DK_manuale
*****
MIGLIOR CONTROLLORE OTTENUTO AL PASSO
      5
CHE REALIZZA UN MU:
      0.7705
*****

```

L'algoritmo è meno efficace di quello che utilizza il comando *musyn*, ma comunque è efficace nella riduzione di $\mu_{\Delta_{aug}}$ rispetto al controllore $K_{hinf_{syn}}$.

In figura 28 è riportato il grafico dell'andamento del valore singolare strutturato per ogni step.

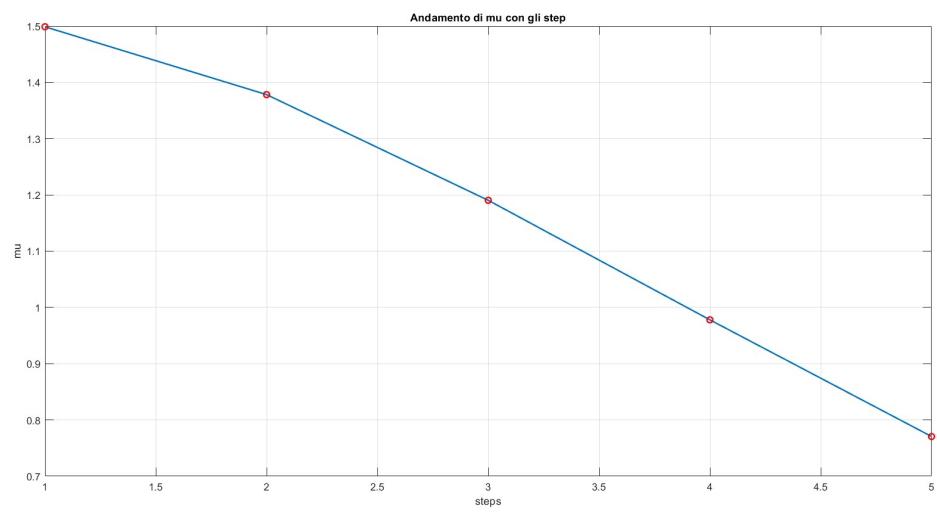


Figura 28: Andamento di μ col proseguire degli step

8.4 Riduzione dell'ordine del controllore DK

Il comando *musyn* genera un controllore di ordine abbastanza elevato⁵ per cui abbiamo tentato di generare controllori di ordine minore utilizzando la tecnica della balanced truncation.

```
% Sintetizziamo anche un controllore di ordine minore

K_DK_balanced = reducespec(K_DK, "balanced");

K_DK_balanced = process(K_DK_balanced);

% [k_dk_sys] = getrom(K_DK_balanced, Order=length(K_hinfsyn.A));
[k_dk_sys] = getrom(K_DK_balanced, Order=(length(K_DK.A)/2));

K_DK_balanced_tf = tf(k_dk_sys);


% Sintetizziamo un controllore di ordine molto basso: 2

[k_dk_sys_2, info] = getrom(K_DK_balanced, Order=2);

K_DK_2_order_tf = tf(k_dk_sys_2);
```

⁵Nel caso in esame ha restituito un controllore di ordine 62